

DataScience Copilot — Interview Guide

Two parts: **Part 1** explains what the project actually does, in plain language you can say out loud. **Part 2** is a bank of likely interview questions with quick hints.

PART 1 — Understand the Project (Easy Version)

In one line

It's an **AI data scientist**. You upload a spreadsheet and type your goal in plain English — like "*predict which customers will churn*" — and the system does the entire machine-learning workflow on its own: it explores the data, cleans it, builds features, trains several models, picks the best one, checks how accurate it is, explains *why* it makes its predictions, and writes a report with a downloadable model. **And if any step breaks, it diagnoses the problem and fixes itself instead of just crashing.**

That last part — the self-fixing — is the most interesting piece.

The one idea that explains everything

The AI decides *what* to do. Plain Python code does *the actual work*.

- The **LLM (the AI brain)** is like a *manager*. It makes decisions: "This looks like a churn problem, the target column is `Churned`, we should log-transform the `income` column."
- The **tools (regular Python code)** are like the *workers*. They do the real ML — cleaning, training, scoring — using trusted libraries (pandas, scikit-learn, XGBoost).

The AI never writes or runs code. It only picks from a fixed menu of approved actions. This makes the system **safe, testable, and reproducible** — you're not letting an AI run random code on your machine.

How it works — a simple analogy

Think of it like an **assembly line with a smart foreman**:

1. **You drop off** a dataset + a goal.
2. A **planner (the AI)** looks at the data and writes the to-do list: profile → clean → build features → train → evaluate → explain → report.
3. But the AI can't invent random steps — it can only pick from a **fixed list of approved stations**, and a **validator** double-checks the order makes sense before anything runs. (You can't "train a model" before you "clean the data.")
4. The line runs each station one at a time, **saving progress after every step** (like checkpoints in a video game — a failed run can resume instead of starting over).

5. At the end you get a **report**, a **leaderboard of models**, an **explanation of what drives the predictions**, and a **downloadable trained model**.

The star feature — the self-fixing loop

This is what makes the project stand out. Explain it with the real example:

Say training crashes with an error like *"could not convert string 'Yes' to float."* Normally your whole pipeline dies. Instead, my system **catches the error**, looks at it, and recognizes the pattern: *"the target column has Yes/No text instead of numbers."* It then **automatically converts Yes/No into 1/0** and **retries that same step** — and now it works. No human needed.

The clever design details:

- **It's smart about *when* to use the AI.** First it tries cheap, instant **pattern-matching rules** to identify the error. Only if it's *unsure* does it ask the AI — and even then with a 20-second timeout, so it can never hang.
- **It separates two kinds of problems:** "fix the data and retry" (encoding errors, missing values) vs. "the plan itself is wrong" (bad model choice). Different problems, handled differently — and that boundary lives in one place, so the logic stays simple.
- **It has budgets.** It only tries a few times before giving up and asking a human, so it can never get stuck in an infinite loop.

Tech stack (say it in one breath)

FastAPI for the backend API, **LangGraph** to run the pipeline as a graph with checkpointing, **pandas / scikit-learn / XGBoost / LightGBM** for the actual machine learning, **SHAP** to explain predictions, and a **Next.js + React** frontend. The AI models are reached through **OpenRouter**, so it's not locked to any single provider.

Why LangGraph? Two reasons: it lets the *step order change per run*, and it *saves progress after each step* so a failed run can resume instead of restarting from scratch.

Three phrases that make you sound sharp

1. **"LLM decides, deterministic tools execute"** — the core design philosophy.
2. **"The graph is static, but the plan is dynamic"** — the steps are pre-built and safe, but the AI chooses which ones to run and in what order.
3. **"Leak-safe by construction"** — all the learning (scaling, encoding) happens *inside* the cross-validation, so the model never accidentally peeks at test data.

Be honest about the limits (this builds credibility)

Saying these *before* the interviewer finds them turns a weakness into a strength:

- **It's synchronous right now** — the API call stays open until the whole run finishes. The fix is a background job queue; I know exactly where that change goes.
- **The "start over" branch isn't fully intelligent yet** — when it can't self-fix, it retries and then

asks a human, rather than cleverly changing strategy. It's a deliberate placeholder.

- **It's single-machine** — checkpoints go to local SQLite and files to local disk; scaling out means moving those to shared storage.

PART 2 — Interview Questions (with quick hints)

Hints point you toward the answer; practice saying each one out loud.

1. Walk me through it

1. **"Walk me through what happens from upload to final report."** Hint: *upload validates by parsing → create run → profile → planner + validator → route walks the plan → deterministic tools + LLM agents → summarize sets completed.*
2. **"Where does the LLM actually get used, and where doesn't it?"** Hint: *LLM in planner + 4 agents (problem / feature / insights / recommendations); everything else is deterministic tools; the LLM never runs code.*
3. **"What decides the order the steps run in?"** Hint: *the Planner proposes names, `validate_plan` checks the DAG, `route` walks the plan by cursor.*
4. **"Show me the single most interesting flow in the system."** Hint: *the `Yes/No` target failure → `TARGET_ENCODING_ERROR` → `label_encode_target` → `retry` → `success`.*
5. **"What does the user get at the end?"** Hint: *`run.json` (plan, reports, reflection history) + downloadable `model.joblib` and `report.md`.*

2. Architecture & design

1. **"Why LangGraph and not a plain function pipeline or a Celery chain?"** Hint: *conditional-edge routing for dynamic order + retry loops, and per-node checkpointing for resume.*
2. **"Why is the graph static but the plan dynamic?"** Hint: *nodes registered once in `build_graph`; `route` + the plan pick the path per run.*
3. **"Why does state hold file paths instead of the DataFrames themselves?"** Hint: *keeps the state and every checkpoint small; heavy data lives on disk as Parquet / joblib.*
4. **"How do you stop the LLM from producing an impossible plan?"** Hint: *registry-bounded names + `validate_plan` DAG check (requires / produces / duplicates / terminal) + feedback-retry.*
5. **"Why separate 'repair the artifact' from 'escalate to the planner'?"** Hint: *data problems are retryable in place; strategy problems need a different plan; `select_repair` enforces one or the other.*
6. **"How does structured output work without a provider's JSON mode?"** Hint: *schema-in-prompt + `model_validate_json` + bounded validation-retry in the client.*
7. **"Why is the OpenAI SDK here if you're not using OpenAI?"** Hint: *it's only the transport to OpenRouter's OpenAI-compatible endpoint; the abstraction is `LLMClient`.*

3. Deep-dive code

1. "What exactly happens when a capability node raises?" *Hint: the wrapper catches it, writes `plan_error` / `failed_capability` / `failed_exc_type`, never re-raises.*
2. "Walk me through the reflect node's decision order." *Hint: check budgets → diagnose → `select_repair` → run repair → if changed retry, else escalate; the cursor never advances.*
3. "How does diagnosis avoid hanging on a slow LLM?" *Hint: heuristic first (≥ 0.6 confidence trusted); LLM fallback in a thread pool under a 20-second timeout; returns the heuristic on timeout.*
4. "How is data leakage prevented during training?" *Hint: only row-wise transforms in the features step; impute / scale / encode live inside the sklearn Pipeline, fitted per fold.*
5. "How are the held-out metrics computed honestly?" *Hint: `evaluate_model` re-splits (test size 0.2), clones the pipeline, refits on train only, scores on the held-out split.*
6. "How does `explain_model` guarantee it never breaks the run?" *Hint: guarded `import shap`; `TreeExplainer` / `LinearExplainer` → permutation fallback → method "unavailable"; never raises.*
7. "How does the `cv_folds` knob repair actually reach the training tool?" *Hint: `reduce_cv_folds` sets the knob; reflect deep-copies it into the spec; training reads `spec.get("cv_folds")`.*

4. Why this, not that (trade-offs)

1. "Why let deterministic tools do the ML instead of asking the LLM to write code?" *Hint: reproducible, testable, no arbitrary code execution; LLM decisions are cheap to validate.*
2. "Why heuristic-first diagnosis instead of always asking the LLM?" *Hint: common failures are cheap / deterministic / offline; the LLM is only for the uncertain tail.*
3. "Why SQLite checkpointing instead of just re-running on failure?" *Hint: resume from the last good node; don't redo expensive training; but it's single-machine (the trade-off).*
4. "Why Parquet for intermediates instead of CSV or in-memory?" *Hint: typed + compact + fast re-reads on retry; repairs rewrite it in place at the same path.*
5. "Why cap recommendations at 6 and retries at small numbers?" *Hint: bounded budgets guarantee termination; the recursion limit is sized above them as a final backstop.*

5. Failure & edge cases

1. "A user uploads a file that isn't really a CSV — what happens?" *Hint: it's parsed with `profile_dataset` before being accepted; parse failure → rejected and deleted, no run created.*
2. "The target column is Yes/No strings and training crashes — then what?" *Hint: caught → `TARGET_ENCODING_ERROR` → `label_encode_target` rewrites to 0/1 → same step retried → completes.*
3. "A repair runs but changes nothing — does it keep retrying forever?" *Hint: no; an "applied AND changed" gate — a no-op escalates instead of burning a retry.*
4. "Reflection can't fix it — is the failure ever intelligently re-planned?" *Hint: honestly, not yet — escalation retries the same plan then stops at `needs_input`; the hint is recorded, not*

acted on.

5. **"The planner LLM keeps emitting an invalid plan — does the request hang or 500?"**

Hint: neither; after 3 attempts it returns `needs_input` with the validator errors; a node exception becomes a `failed` run record, not an HTTP 500.

6. Scaling & production

1. **"This blocks the HTTP request for the whole run — how would you fix that?"** *Hint: background worker / job queue; `POST /runs` returns an id immediately; poll `GET /runs/{id}`.*

2. **"How would you run this across multiple machines?"** *Hint: move checkpoints to a shared DB and artifacts to object storage (today: SQLite + local disk).*

3. **"A dataset is 5 GB — where does it fall over, and what changes?"** *Hint: everything loads into pandas in memory; need chunking / out-of-core or a sampling step; upload is capped at 200 MB today.*

4. **"How would you observe this in production?"** *Hint: the reflection history is already an audit trail; add metrics on repair frequency, escalation rate, per-node latency.*

5. **"How would you make the escalation branch actually useful?"** *Hint: act on the planner hint — model swap, shrink search space, edit the plan — the seam is already there.*

7. Reflection

1. **"What's the part you're most proud of, and why?"** *Hint: the diagnose → repair → retry loop with a clean repair-vs-escalate boundary and bounded budgets.*

2. **"What would you rebuild if you started over?"** *Hint: async runs from day one; and make escalation act on the hint instead of just retrying.*

3. **"What surprised you or was harder than expected?"** *Hint: keeping the self-fix loop from looping forever — the budgets + recursion-limit backstop exist for exactly that.*

4. **"If you had one more week, what's the highest-value thing to add?"** *Hint: a background job queue (unblocks the API) or a real replan on escalation — pick one and justify it.*